Intégration d'un système de messagerie temps réel dans une plateforme SaaS B2B

Projet d'Étude Professionnel — Concepteur Développeur Web Full Stack

Candidat	Hedi Boissard
Entreprise	VoxTrend
Période	Février – Juillet 2026
Tuteur	Alexandre Rangom, CTO & Co-fondateur

SLIDE 2 SOMMAIRE

Plan de la présentation

01 Présentation de VoxTrend & du produit

02 Contexte, mission & environnement technique

03 Problématique & analyse des besoins

04 Solution technique retenue

05 Architecture & implémentation

06 Réalisations concrètes

07 IA dans le workflow de développement

08 Analyse critique & recommandations

09 Bilan, compétences & conclusion

SLIDE 3 PRÉSENTATION DE VOXTREND & DU PRODUIT

VoxTrend — Qui sommes-nous ?

VoxTrend est une startup SaaS B2B fondée en 2024, spécialisée dans l'analyse et l'activation des interactions clients. La plateforme centralise les avis, messages, e-mails et interactions sur les réseaux sociaux pour permettre aux entreprises de mieux comprendre et activer leurs clients.

Informations clés

- Secteur — SaaS B2B / Customer Intelligence
- Taille de l'équipe — 7–8 personnes
- Localisation — Paris 2ème (Palais Brogniart)
- Co-fondateurs — Issa Hadjidj (CEO) et Alexandre Rangom (CTO)

Fonctionnalités clés de la plateforme

- **Agrégation multicanal** — avis, e-mails, réseaux sociaux
- **Analyse automatisée** — sentiments et tendances
- **Dashboards** — suivi en temps réel
- **Review Booster** — workflows personnalisés pour automatiser les campagnes de récolte d'avis
- **Widget chatbot** — intégrable pour e-commerçants et retailers

Clients cibles & stack technique

E-commerçants et retailers souhaitant centraliser la gestion de leurs interactions clients.

Stack : Next.js (App Router) · NestJS · TypeScript · PostgreSQL / TinyBird · Tailwind CSS · Radix UI · shadcn/ui

SLIDE 4 CONTEXTE, MISSION & ENVIRONNEMENT TECHNIQUE

Mon rôle : Développeur Fullstack Next.js & IA

Intégré au sein de l'équipe technique de VoxTrend, j'ai participé au développement du front-end de la plateforme SaaS B2B, avec une responsabilité particulière sur le système de messagerie temps réel, la page pricing, les pages settings et le widget chatbot intégrable.

Organisation de l'équipe

- **Collaboration directe** — avec Alexandre Rangom (CTO) et Mohammed Gueye (développeur)
- **Communication quotidienne** — via groupe Slack dédié
- **Suivi des tâches** — documentation sur Notion
- **Point quotidien** — avec le CTO chaque matin pour aligner les priorités

Stack & environnement technique

Couche	Technologies
Frontend	Next.js 14+ (App Router), React, TypeScript, shadcn/ui, Radix UI, Tailwind CSS, Zustand
Backend	NestJS (Node.js), PostgreSQL, TinyBird, REST APIs, Ably (WebSockets)
Outils & workflow	Git / GitHub, Cursor, Claude Code, OpenCode, Notion

Environnement de travail : Présentiel — Paris 2ème, Palais Brogniart

SLIDE 5 PROBLÉMATIQUE & ANALYSE DES BESOINS

> *« Comment intégrer un système de messagerie temps réel fiable, scalable et maintenable au sein d'une plateforme SaaS B2B en production ? »*

Contexte

VoxTrend avait besoin d'une fonctionnalité de messagerie instantanée permettant aux équipes clients de ses utilisateurs de communiquer en temps réel depuis la plateforme. Cette fonctionnalité devait s'intégrer sans friction dans l'architecture existante, tout en répondant aux exigences de performance et de fiabilité d'un produit SaaS en production.

Enjeux identifiés

- **Technique** — Choisir et intégrer la bonne technologie temps réel dans une architecture Next.js App Router
- **Produit** — Garantir une expérience utilisateur fluide et réactive, sans latence perceptible
- **Organisationnel** — Livrer dans les délais contraints d'un environnement startup

Besoins fonctionnels

- Envoi et réception de messages en temps réel
- Gestion de plusieurs canaux de discussion simultanés
- Notifications push et in-app via Ably
- Gestion du cache et des server actions avec Next.js
- Persistance des messages avec historique consultable

Contraintes

- **App Router** — Compatibilité avec l'architecture Next.js App Router (server components par défaut)
- **Zustand** — Gestion de l'état global sans complexité supplémentaire
- **Fiabilité** — Reconnexion automatique en cas de coupure réseau
- **Délai** — 2 semaines pour la première version fonctionnelle

SLIDE 6 SOLUTION TECHNIQUE RETENUE

Quelle technologie choisir ?

Solution	Points forts	Limitations
Socket.IO	Rooms natives, reconnexion intégrée	Serveur WebSocket séparé + Redis pour scaling
Supabase Realtime	Sync directe depuis Postgres	Latence : messages transitent par la BDD
Pusher	Service managé, simple	Pricing, moins de contrôle
Ably ✓ RETENU	SDK Next.js, reconnexion auto, scaling natif	Coût service tiers
WebSockets natifs	Contrôle total	Tout à implémenter soi-même

Choix retenu — Ably + Zustand

✓ Focus sur la logique métier, pas sur l'infrastructure
✓ Reconnexion automatique et scaling gérés nativement
✓ SDK officiel compatible Next.js App Router
✓ Zustand : store léger, sans boilerplate, intégration naturelle avec App Router

Ably WebSockets — Realtime-as-a-Service

Ably est une plateforme de messagerie temps réel en tant que service. Elle offre une infrastructure mondiale hautement disponible, avec des garanties de livraison des messages et une reconnexion automatique.

Chiffres clés Ably

Métrique	Valeur
Latence mondiale moyenne	~37ms (médiane round-trip : 46ms)
Disponibilité SLA	99.999% (~5min 15s de downtime/an)
Infrastructure	7 datacenters + 635 points de présence

SLIDE 7 ARCHITECTURE & IMPLÉMENTATION

Vue d'ensemble — Flux de données

L'architecture de la messagerie repose sur quatre couches distinctes qui communiquent entre elles via des protocoles adaptés à chaque besoin.

#	Couche	Description
1	Utilisateur → Next.js	L'utilisateur envoie un message depuis l'interface Next.js
2	Next.js → NestJS	Appel REST vers NestJS qui valide et persiste en base de données
3	NestJS → Ably	NestJS publie le message sur le channel Ably via l'API REST
4	Ably → Clients	Ably distribue le message à tous les abonnés via WebSocket
5	Ably → Zustand	Le frontend reçoit le message et met à jour le store Zustand
6	Zustand → UI	L'interface se met à jour instantanément sans rechargement

Étapes de développement

- **1.** Configuration Ably — Initialisation du client, gestion des clés API en environnement sécurisé
- **2.** Gestion des canaux — Création dynamique des channels par conversation
- **3.** Abonnements — Subscribe aux événements de nouveaux messages côté client
- **4.** Publication — Envoi des messages via NestJS (sécurisation serveur)
- **5.** État global — Synchronisation des messages dans le store Zustand
- **6.** Persistance — Sauvegarde en base PostgreSQL via NestJS

Défis techniques rencontrés

- **Server vs Client Components —** Le SDK Ably nécessite une connexion WebSocket côté navigateur ("use client"). Solution : isoler la logique temps réel dans des client components dédiés tout en gardant le layout côté serveur.
- **Synchronisation à la reconnexion —** Détecter les événements de reconnexion, effectuer un appel REST pour récupérer les messages manquants, puis fusionner avec l'état local en évitant les doublons.
- **Sécurisation des tokens —** Le frontend demande un token temporaire au backend NestJS. Le backend génère un token signé avec des capabilities restreintes, renouvelé automatiquement avant expiration.

SLIDE 8 RÉALISATIONS CONCRÈTES

Chatbot intégrable — De la messagerie à l'IA

Développement d'un widget de chat intégrable en deux lignes de code sur n'importe quel site e-commerce, créant un canal de communication direct entre les visiteurs et les gérants via la plateforme VoxTrend.

Ce que j'ai construit

- **Widget côté visiteur** — accueil personnalisé, questions fréquentes, historique des conversations, chat en temps réel
- **Dashboard côté agent** — interface dédiée dans VoxTrend avec liste des conversations, notifications et système de tickets
- **Pont temps réel** — connexion bidirectionnelle via Ably entre widget et dashboard, création automatique de tickets

État actuel & évolutions

- **Statut —** Fonctionnel en local, intégration dans la plateforme en cours
- **Prochaine étape —** Intégration LLM pour réponses automatiques aux questions courantes (support hors horaires)
- **Vision —** Synthèse automatique des conversations via IA

Composants UI & Dashboards

Composants développés et documentés avec shadcn/ui, Radix UI et Tailwind CSS, garantissant la cohérence visuelle sur l'ensemble de la plateforme.

Composant	Description
TicketPanel	Panneau de gestion des tickets avec historique, infos visiteur et actions rapides. Synchronisé via Ably et Zustand.
WorkflowNodes	Éditeur visuel de workflows pour le Reviews Booster. Drag & drop, connexions entre nœuds.
ApiConnectionCard	Carte de connexion aux services externes (Google, Meta) avec statut temps réel.
SubscriptionCard	Carte d'abonnement pour la page pricing avec gestion des états actif/inactif.

Principes appliqués

- Accessibilité (ARIA, navigation clavier via Radix UI)
- Responsive design mobile-first avec Tailwind CSS
- Design system cohérent avec support du dark mode

SLIDE 9 IA DANS LE WORKFLOW DE DÉVELOPPEMENT

Outils IA utilisés

Outil	Usage
Cursor	IDE avec IA intégrée — autocomplétion contextuelle, refactoring, génération de composants
Claude Code	Tâches complexes, revue de code, documentation, débogage d'architectures
OpenCode	Interface terminal pour des interactions rapides avec l'IA sur des problèmes ponctuels

Impact concret mesuré

- **WorkflowNodes —** Généré en 20 min au lieu de 3h estimées (drag & drop)
- **Bug Zustand/Ably —** Identifié en quelques minutes via Claude Code
- **Gain de temps —** Réduction significative sur les tâches répétitives : boilerplate TypeScript, typage des props

> *Nuance critique : L'IA accélère le développement mais ne remplace pas la réflexion architecturale. Chaque suggestion doit être comprise et validée — le développeur reste le décideur final. L'IA est un accélérateur, pas un substitut.*

SLIDE 10 **ANALYSE CRITIQUE & RECOMMANDATIONS**

Retour d'expérience

Ce qui a bien fonctionné

✓ Ably : reconnexion automatique et channels natifs → livraison rapide sans gérer l'infra temps réel
✓ Zustand a simplifié la synchronisation entre composants et réduit le boilerplate vs Redux
✓ Séparation server/client components : architecture maintenable avec responsabilités bien définies

Difficultés rencontrées

- Complexité du découpage server/client components avec WebSockets : plusieurs itérations nécessaires
- Persistance et synchronisation temps réel : cas limites difficiles à anticiper (doublons, messages manquants)
- Absence de tests automatisés au démarrage : débogage plus coûteux en temps

Ce que je ferais différemment

- Mettre en place des tests d'intégration dès le début pour les flux temps réel
- Documenter l'architecture technique en amont avec des diagrammes de flux
- Prévoir un environnement de staging pour tester les scénarios de reconnexion

Recommandations pour VoxTrend

- Pipeline CI/CD avec tests automatisés sur les flux critiques de messagerie
- Monitoring des performances temps réel (latence, taux de reconnexion) en production

SLIDE 11 BILAN, COMPÉTENCES & CONCLUSION

Compétences techniques acquises

- Maîtrise de Next.js App Router (server/client components, routing, layouts, server actions)
- Intégration de systèmes temps réel avec Ably WebSockets
- Architecture frontend avec Zustand et patterns de gestion d'état réactif
- Développement backend avec NestJS et conception d'APIs REST
- Composants UI accessibles et réutilisables (shadcn/ui, Radix UI)
- Utilisation professionnelle des outils IA (Cursor, Claude Code) au quotidien

Compétences transversales

- Autonomie progressive dans un environnement startup, prise en charge de fonctionnalités de bout en bout
- Communication technique quotidienne avec un CTO et un développeur senior
- Livraison dans des délais contraints tout en maintenant la qualité production
- Gestion des priorités et arbitrage entre rapidité et robustesse technique

Synthèse & perspectives

Ce PEP m'a permis de répondre concrètement à la problématique posée : intégrer un système de messagerie temps réel robuste dans une plateforme SaaS B2B est réalisable avec les bons outils — à condition d'anticiper les contraintes architecturales de Next.js App Router et de choisir une solution adaptée au contexte startup.

Perspectives d'évolution

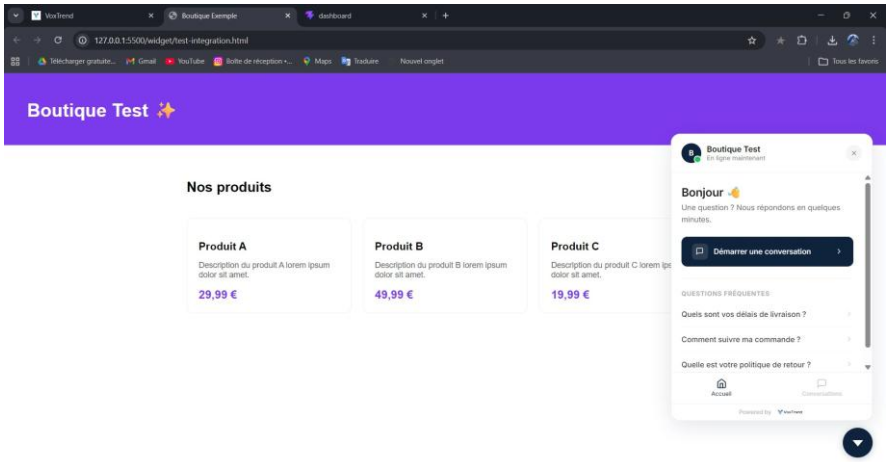
- **Intégration IA dans le chatbot —** Appels LLM pour automatiser les réponses et générer des synthèses
- **Notifications push multicanal —** Push navigateur, e-mail et intégration mobile
- **Analytics conversationnels —** Insights automatisés : temps de réponse, satisfaction client, sujets récurrents

> *« Construire une feature de messagerie temps réel dans un produit SaaS en production, c'est comprendre que la robustesse technique et l'expérience utilisateur sont indissociables. »*

SLIDE **ANNEXES**

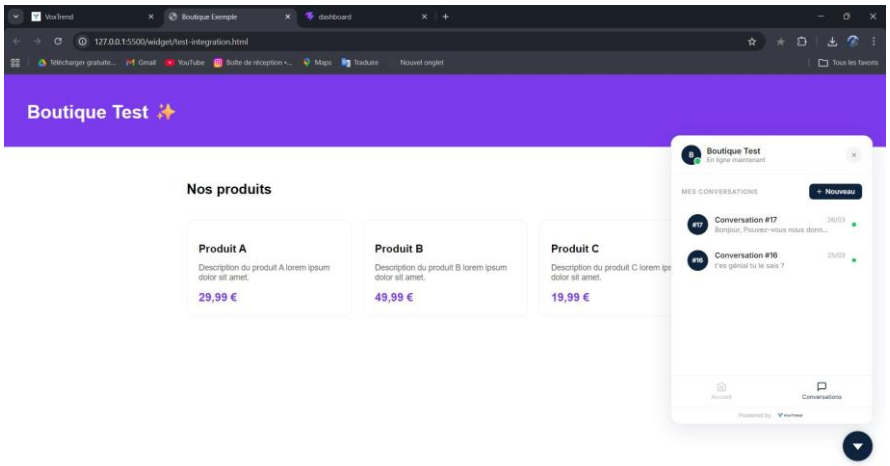
Captures d'écran de la plateforme VoxTrend

Annexe 1 — Chatbot — Widget côté visiteur (accueil)



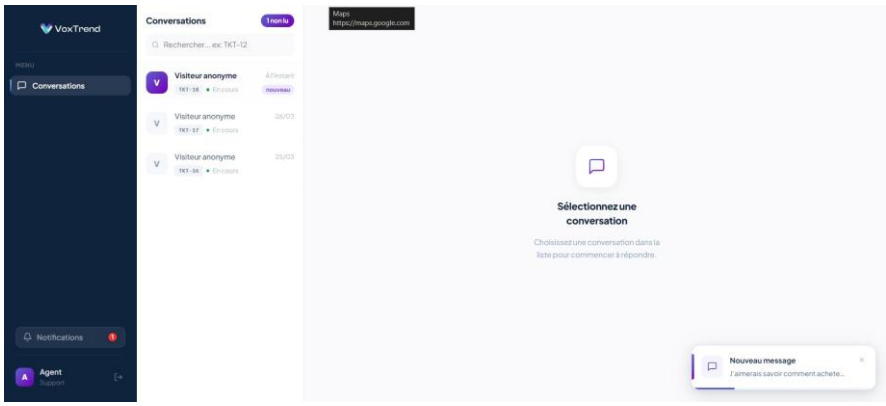
Page d'accueil du widget avec le nom de la boutique, le statut « En ligne », le message de bienvenue et les questions fréquentes préconfigurées.

Annexe 2 — Chatbot — Widget côté visiteur (conversations)



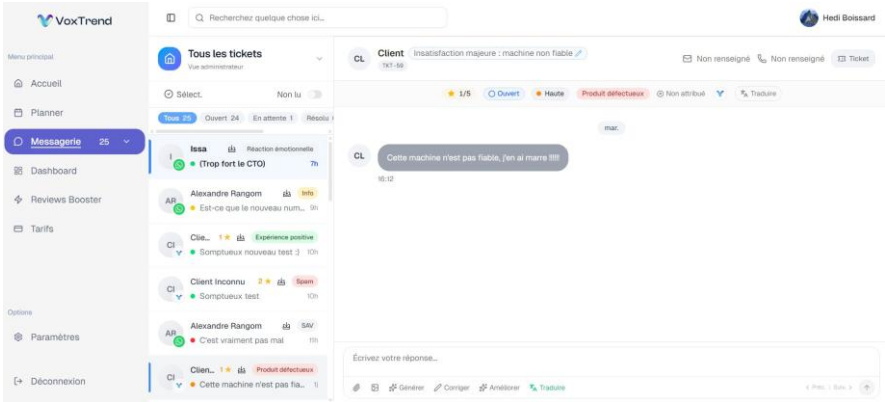
La vue « Conversations » du widget. Le visiteur retrouve l'historique de ses échanges avec la boutique.

Annexe 3 — Dashboard agent (côté VoxTrend)



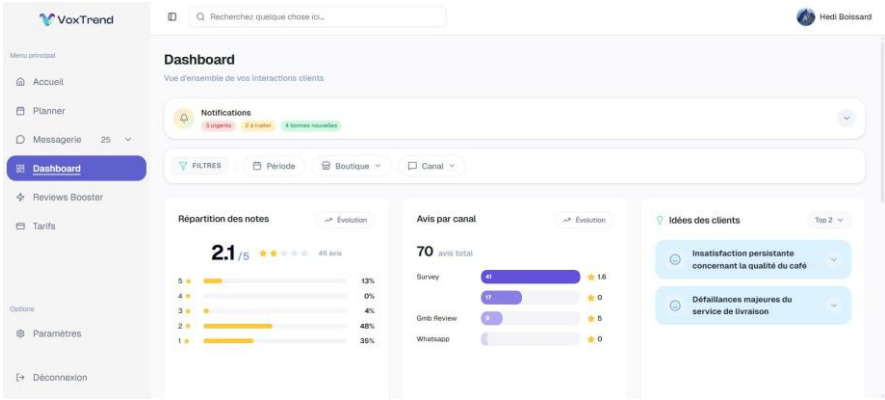
L'interface agent dans VoxTrend avec liste des visiteurs, statuts des tickets et notifications en temps réel.

Annexe 4 — Messagerie (TicketPanel)



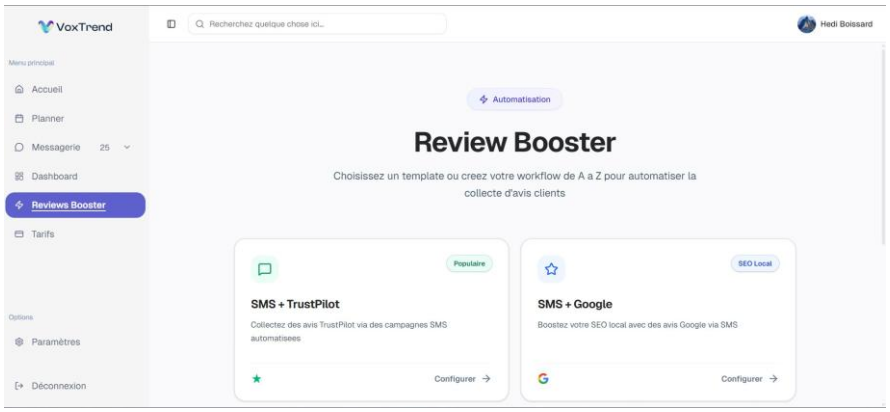
Interface de messagerie avec liste des tickets, indicateurs de sentiment et badges de catégorisation automatique.

Annexe 5 — Dashboard principal



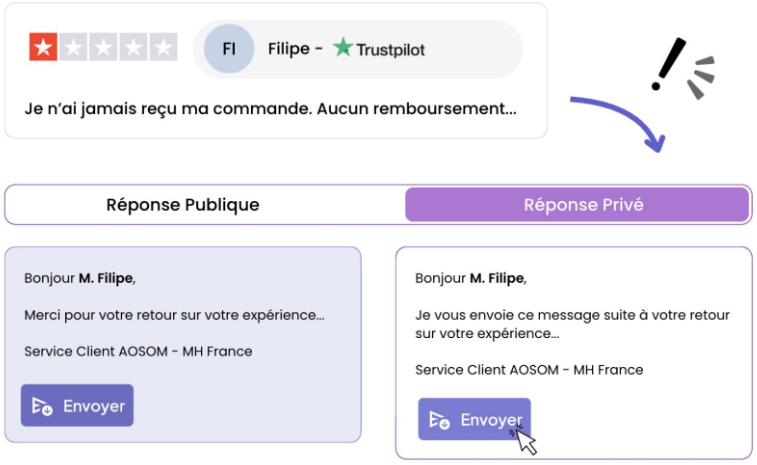
Vue d'ensemble des interactions clients avec répartition des notes et idées clés extraites par l'IA.

Annexe 6 — Reviews Booster



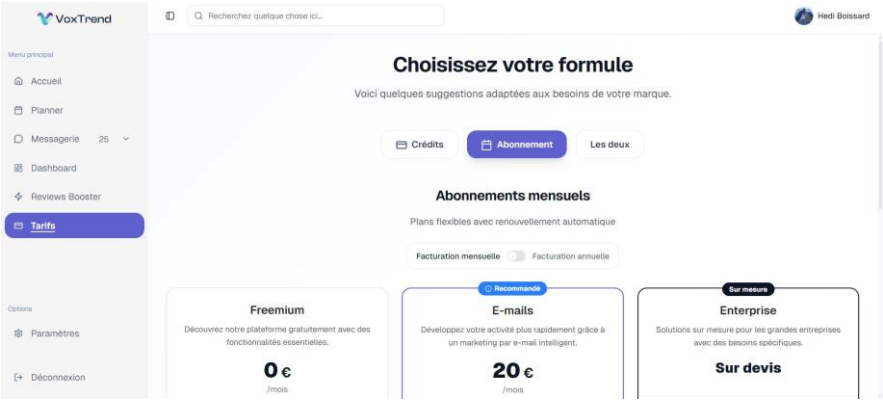
Page de sélection des templates de workflows automatisés pour la collecte d'avis clients.

Annexe 7 — Système de réponse aux avis



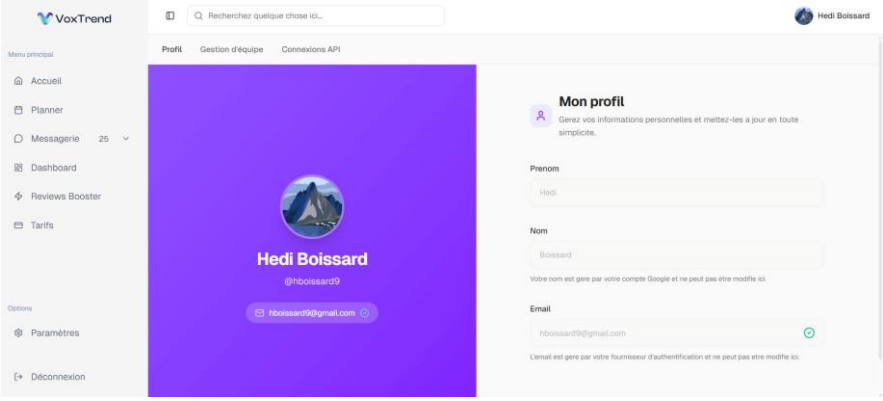
Interface de réponse aux avis clients avec double mode (publique et privée). Réponses pré-générées par l'IA.

Annexe 8 — Page Tarifs (SubscriptionCard)



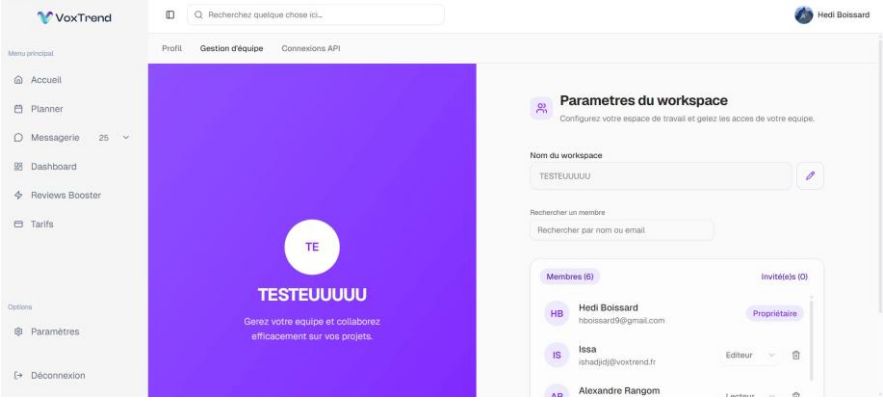
Page de pricing avec les différentes formules, toggle mensuel/annuel et modes Crédits/Abonnement.

Annexe 9 — Page Profil (Settings)



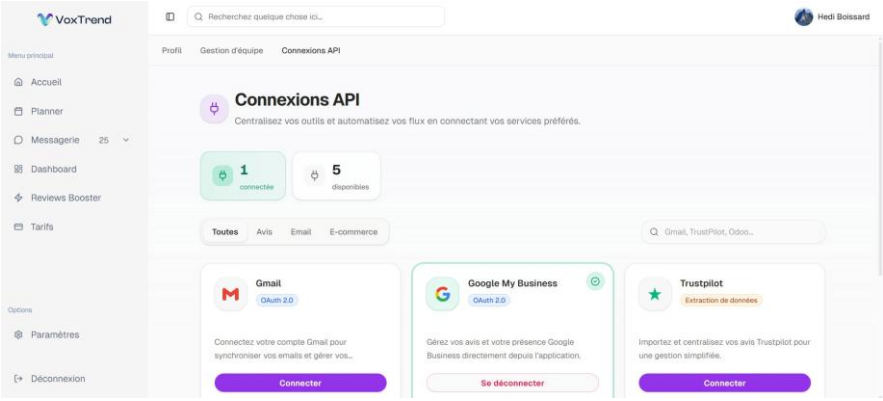
Page de gestion du profil utilisateur avec les onglets Profil, Gestion d'équipe et Connexions API.

Annexe 10 — Gestion d'équipe (Settings)



Interface de gestion du workspace avec la liste des membres, leurs rôles et les actions d'invitation.

Annexe 11 — Connexions API (ApiConnectionCard)



Page de gestion des intégrations tierces avec cartes de connexion (Gmail, Google My Business, TrustPilot).